PDEA'S
PROF. RAMKRISHNA MORE ART'S, COMMERCE & SCIENCE COLLEGE, AKURDI, PUNE - 44
(AUTONOMOUS)

Academic Year 2024-25

(Super-Market Billing System)

Submitted by

Shruti Kamble

Sakshi Mane

Head Of Department

Mrs. Archana Tank

Guided by

Mrs. Smita Pardeshi

Department

SY.BBA-CA (SEM - IV)

CERTIFICATE

This is certify that is Shruti Jitendra Kamble and Sakhshi Sahebrao Mane of the class TYBBA(CA) has satisfactorily completed the project title “Super-Market Billing system ” on Partial fulfilment of the B.B.A(Computer Application) semester-5 (Autonomous) during academic 2025-2026 and this project represented Bonafide work.

Mrs. Jayshree Ma’am

(Project Guide)

Mrs. Archana Tank

(Head of the Department)

Internal examiner

External examiner

INDEX

INTRODUCTION -

Supermarket Billing System
(Point-of-Sale Simulator)

The project titled “Supermarket Billing System” (Modern POS & Billing Automation) is a web-based application developed to streamline and automate the billing process in retail environments such as supermarkets and small stores. In today’s fast-paced retail sector, manual billing or disparate tools can be inefficient, error-prone, and difficult to audit. Traditional methods often lead to slow checkouts, calculation mistakes, and challenges in generating accurate invoices. To overcome these limitations, the Supermarket Billing System provides a robust, end-to-end solution for product entry, cart management, tax computation, digital payments, and instant bill delivery.

The system leverages modern web technologies alongside a lightweight Python backend (Flask) to provide an intuitive cashier interface. Products can be added either by entering a product code manually or by scanning barcodes in real-time using the integrated in-browser Quagga2 scanner. Once added, items are maintained in a session-based cart, which computes subtotal, taxes (GST/VAT), and the final total with precision. The system supports multiple payment modes, including cash and UPI, generating deeplinks and QR codes for seamless digital transactions. Additionally, bills can be delivered directly to customers via WhatsApp using pywhatkit or Twilio APIs, and a professional PDF invoice can be generated on demand for record-keeping and sharing.

One of the distinguishing features of this system is its responsive and user-friendly frontend, which provides clear visual feedback for cart updates, payment status, and invoice generation. By centralizing calculations and business logic in the server, the system ensures consistency and reduces the risk of human errors. This makes the Supermarket Billing System not only a practical tool for small-scale retail operations but also an educational demonstration of web development, session management, and POS workflows.

From a practical perspective, this project highlights the integration of multiple technologies—barcode scanning, digital payments, automated messaging, and PDF generation—to create a cohesive retail solution. It is designed to be modular and extensible, allowing future enhancements such as persistent inventory databases, user authentication, discount engines, and multi-register synchronization. The offline-first approach ensures that the system remains functional even in environments with limited internet connectivity, making it suitable for a wide range of deployment scenarios.

The Supermarket Billing System demonstrates how modern software design can improve operational efficiency, reduce administrative workload, and enhance customer experience. Its modular architecture and clear interface make it adaptable for small stores, training purposes, and further expansion into a fully-featured production-grade POS solution.

ACKNOWLEDGEMENT -

I extend my heartfelt gratitude to all those who have contributed to the successful development of the **Supermarket Billing System**—a digital platform dedicated to streamlining retail billing operations through fast, accurate, and technology-driven solutions.

First and foremost, I would like to express my sincere appreciation to **Prof. Bharti Patenge Madam and Prof. Smita Madam** for their invaluable guidance and constant support throughout this journey. Their expertise, encouragement, and constructive feedback have been instrumental in shaping the system's architecture, user interface, and core functionalities.

I am also deeply grateful to previous projects, open-source tools, and academic references, which served as essential sources of knowledge and inspiration. The insights gained from earlier research and POS system implementations significantly contributed to refining the vision, features, and implementation strategy of this system.

A special thanks goes to various online resources, including **ChatGPT, Stack Overflow, GitHub, YouTube, MDN Web Docs, Visual Studio Code, and Chrome/Edge browsers**, which proved extremely helpful during research, development, and testing phases. These platforms provided technical guidance, innovative ideas, and efficient workflows that enabled the successful execution of the project.

Furthermore, I would like to acknowledge the continuous encouragement and valuable feedback from my peers, friends, and well-wishers.

Lastly, my deepest gratitude goes to the retail and academic communities whose challenges inspired the creation of this project. The **Supermarket Billing System** is a small but meaningful step toward blending modern technology with everyday retail operations, ensuring speed, accuracy, and customer satisfaction in billing processes.

Thank you!

SCOPE OF THE SYSTEM -

The **Supermarket Billing System** is a modern, web-based point-of-sale platform designed for retail environments such as supermarkets, small stores, and training/demo setups. The system integrates manual code entry, live barcode scanning, cart management, digital payments, and instant invoice generation to ensure fast, accurate, and reliable billing operations. Its goal is to eliminate manual calculation errors, speed up checkout, and provide a seamless digital experience for both cashiers and customers.

Product Entry & Live Barcode Scanning –
The system allows quick product addition either through manual code entry or live barcode scanning using an in-browser camera interface. Real-time product lookup from a JSON catalog ensures accurate pricing, category display, and validation, minimizing errors at the point of sale.

Cart Management & Dynamic Updates –
Cashiers can add, update, or remove items from a session-based cart. Quantity adjustments, line totals, subtotals, and taxes (GST/VAT) are computed live, ensuring that the final bill is always accurate. Cart updates are reflected instantly in the user interface, making the checkout process smooth and efficient.

Billing & Tax Calculation –
The system computes subtotal, tax, and grand total for each transaction. The centralized backend ensures consistent calculation across sessions, reducing discrepancies and supporting transparent retail operations.

Payments & UPI Integration –
Customers can pay via cash or UPI. For digital payments, the system generates a UPI deeplink and QR code for seamless scan-and-pay functionality. This feature reduces handling errors, speeds up payment collection, and integrates modern digital payment workflows into the billing process.

WhatsApp Bill Delivery & Invoice PDF –
Bills can be sent directly to customers via WhatsApp using pywhatkit or Twilio APIs. Additionally, the system can generate a professional PDF invoice, including itemized lists, totals, tax, and timestamps, which can be downloaded or shared as needed.

User-Friendly Interface –
The web-based frontend provides a clean, responsive interface with clear visual feedback for product entry, cart updates, payment status, and invoice generation. This design ensures that even new cashiers can operate the system efficiently without extensive training.

Offline & Local-First Functionality –
The application works primarily offline using session storage and local JSON data, making it suitable for stores with limited or intermittent internet access. This ensures uninterrupted operation and reliability in real-world retail settings.

Data Accuracy & Auditability –
All calculations, totals, and transaction logs are handled server-side, ensuring consistency and reducing human errors. PDF invoices and WhatsApp messages provide an auditable record of each transaction.

Future Integration & Scalability –
The system is designed to support future enhancements, including:

- Persistent inventory database (SQLite/PostgreSQL) for real stock management
- User accounts and role-based access for cashiers and managers
- Discounts, promotions, and refund management
- Multi-register synchronization and consolidated reporting
- Mobile-friendly interface and cloud deployment for remote access

REQUIREMENT ANALYSIS -

1. Product Entry & Lookup

- Input product code manually or scan barcode via webcam.
- Lookup product details from JSON database (name, category, price).
- Validate product existence; handle unknown codes gracefully with error feedback.

2. Cart Management

- Add items to cart with default or specified quantity.
- Update quantity; prevent zero or negative values.
- Remove items from cart as needed.
- Maintain a session-based cart for isolation between different cashiers or browsers.

3. Billing & Tax Calculation

- Compute line totals, subtotal, tax (configurable GST/VAT rate), and final total.
- Return totals from backend to ensure consistency and accuracy.
- Display totals dynamically in the UI for cashier reference.

4. Payment Handling (Cash/UPI)

- Cash payments: mark transaction as paid and finalize the bill.
- UPI payments: generate deeplink (`upi://pay`) and QR code for scan-and-pay.
- Include UPI details in WhatsApp messages for customer convenience.

5. WhatsApp Bill Delivery

- Use pywhatkit for local WhatsApp Web messaging (demo mode).
- Optional Twilio integration for programmatic WhatsApp Business messaging.
- Fallback mechanism: simulate sending and log message if APIs are unavailable.

6. Invoice PDF Generation

- Generate on-demand professional PDF invoice with itemized list, subtotal, tax, and total.
- Include timestamp and transaction details for auditing.
- Provide shareable link or downloadable file.

7. User Interface (UI)

- Responsive and intuitive web-based GUI.
- Real-time cart updates, product feedback, and payment status.
- Integrated Quagga2 scanner for continuous barcode scanning with duplicate prevention.
- Action buttons for checkout, WhatsApp sending, and PDF generation.

8. Performance

- Real-time cart updates and barcode scanning with minimal latency.
- Backend calculations optimized for speed and reliability.

9. Security & Data Integrity

- Session-based cart ensures temporary isolation.
- Validate input codes and quantities to prevent errors.

- Logs maintained for transactions and message sending.

10. Scalability & Extensibility

- Designed to integrate persistent inventory databases (SQLite/PostgreSQL).
- Supports future features: discounts, refunds, multi-register, user roles.
- Modular backend allows easy integration with cloud services or mobile apps.

11. Reliability & Fallbacks

- Barcode scanning ensures product identification even if manual entry fails.
- UPI and WhatsApp fallback mechanisms guarantee smooth checkout in various conditions.

12. Usability

- Clear UI for cashiers with minimal training required.
- Status feedback for cart updates, payments, and invoice generation.
- Error messages and confirmations to prevent mistakes during checkout.

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements -

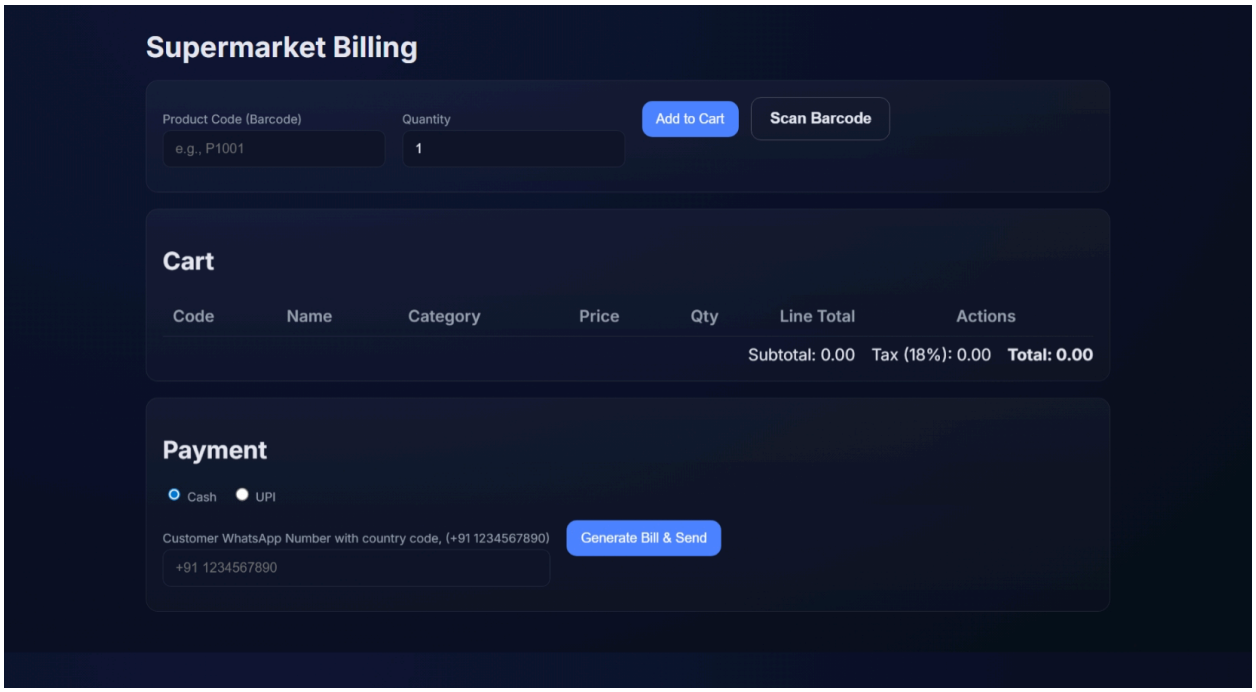
- **Processor**: Intel i5/i7 (8th Gen or later) / AMD Ryzen 5 or higher
- **RAM**: Minimum 4 GB (8 GB recommended for smooth performance)
- **Storage**: At least 256 GB SSD (for faster DB and image processing)
- **Operating System**: Windows 10/11 (64-bit), Linux (Ubuntu), or macOS
- **Camera**: Built-in / External USB Webcam (HD quality recommended)
- **Graphics Card**: Optional, but NVIDIA GPU recommended for faster face recognition (CUDA support with dlib)
- **Network Adapter**: Standard Ethernet/Wi-Fi (system can work offline as well)
- **Backup Storage**: External HDD/Cloud backup (for attendance logs & exports)

Software Requirements -

- **Programming Language**: Python 3.8 or higher
- **IDE/Editor**: Visual Studio Code / PyCharm / Jupyter Notebook (optional)
- **Database**: SQLite (default), MySQL/PostgreSQL (optional for scalability)
- **Libraries & Frameworks**:
    - `opencv-python` (camera & vision)
    - `face_recognition` (face detection & encoding)
    - `numpy` (computations)
    - `pandas` (data manipulation)
    - `openpyxl` (Excel export)
    - `pyzbar` (barcode/QR scanning)
    - `Pillow` (image processing)
    - `tkinter` (GUI)

SYSTEM DESIGN -

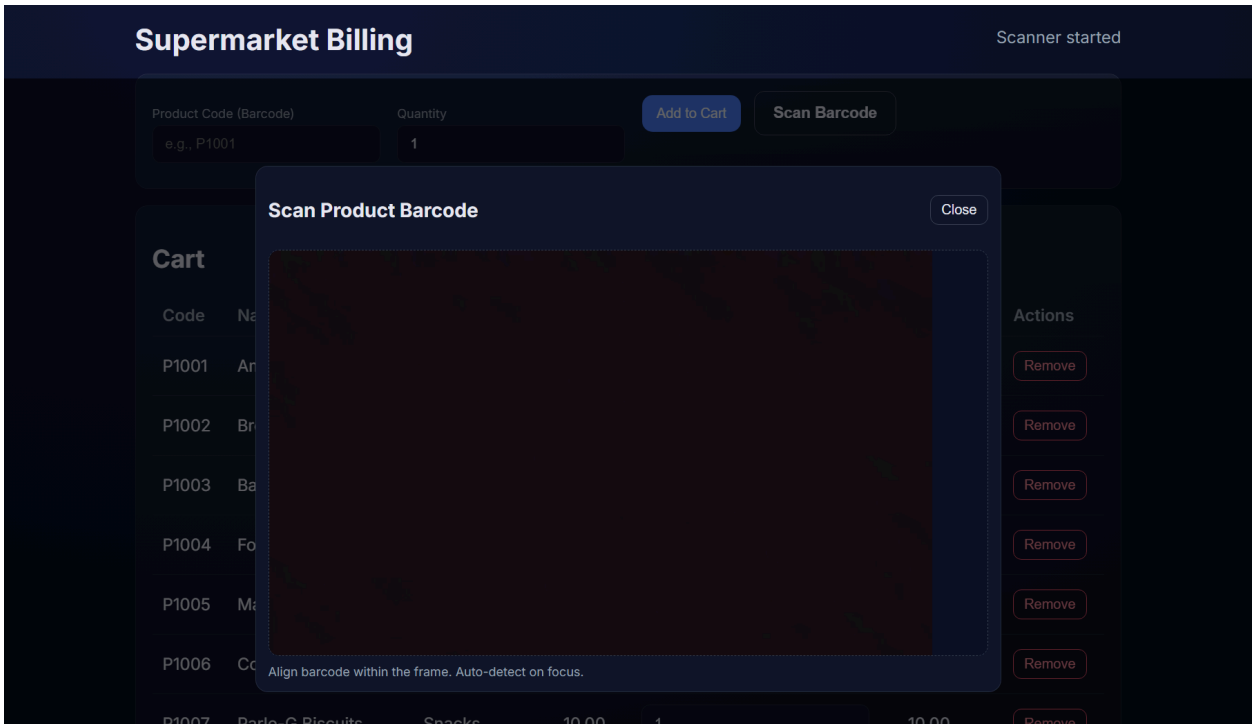
- **Home Page -**



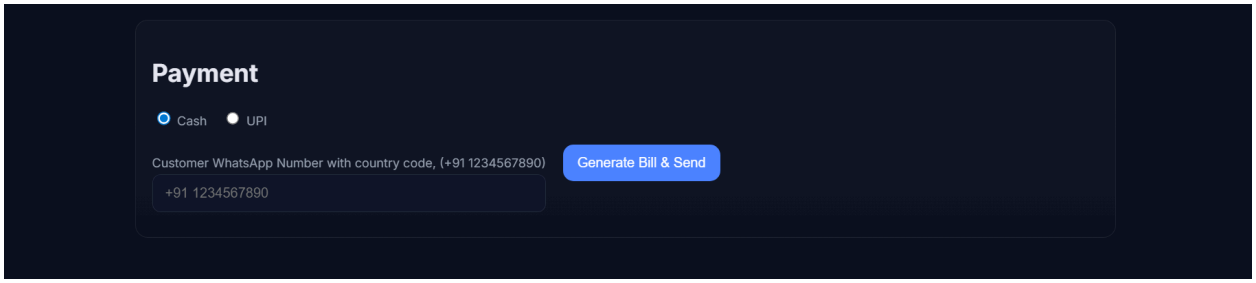
- **Cart -**


Cart
Code Name Category Price Qty Line Total Actions
P1001 Amul Milk 1L Dairy 60.00 1 60.00 Remove
P1002 Bread Loaf Bakery 40.00 1 40.00 Remove
P1003 Basmati Rice 1kg Grains 120.00 1 120.00 Remove
P1004 Fortune Oil 1L Grocery 150.00 1 150.00 Remove
P1005 Maggie Noodles Snacks 15.00 1 15.00 Remove
P1006 Coca Cola 1.25L Beverages 45.00 1 45.00 Remove
P1007 Parle-G Biscuits Snacks 10.00 1 10.00 Remove
P1008 Dove Soap Personal Care 35.00 1 35.00 Remove
P1009 Colgate Toothpaste Personal Care 95.00 1 95.00 Remove
P1010 Tomatoes 1kg Produce 30.00 1 30.00 Remove
Subtotal: 600.00 Tax (18%): 108.00 Total: 708.00

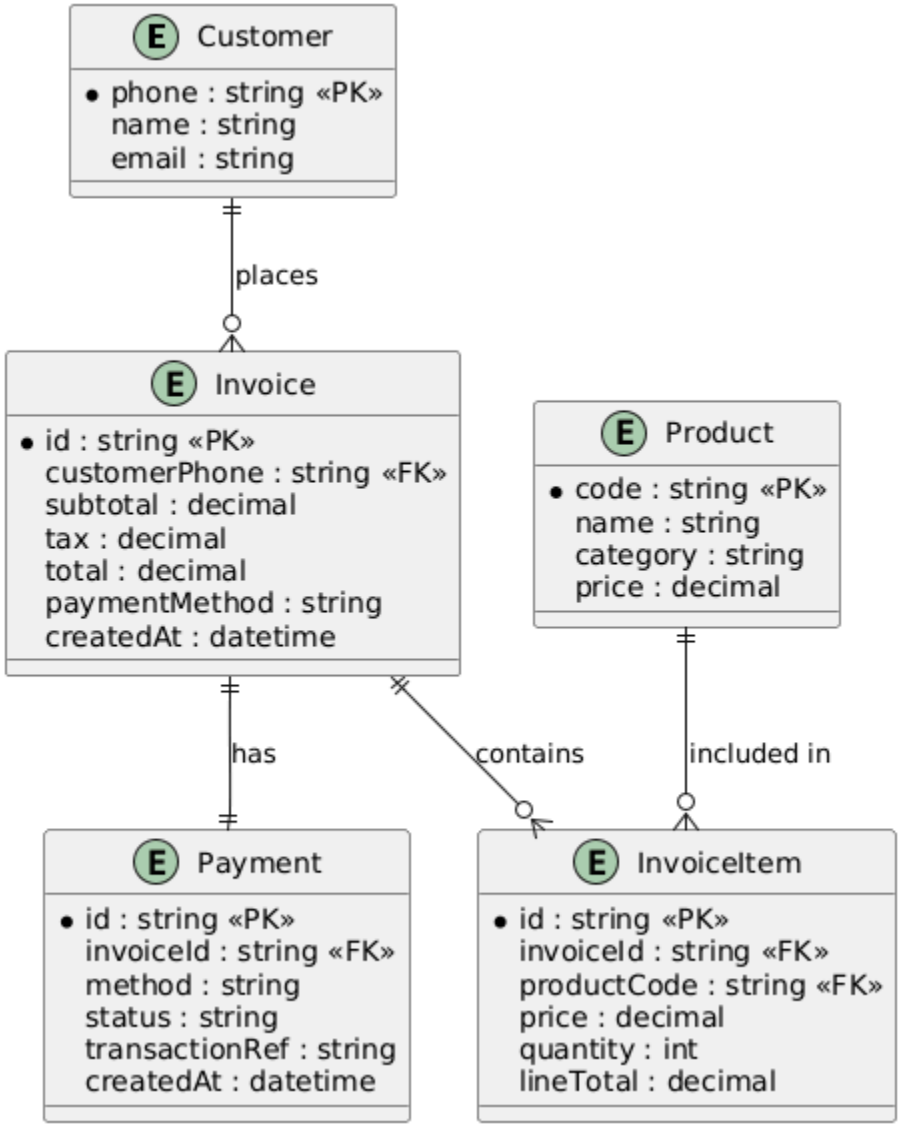
- **Barcode scanner -**



- **Payment -**

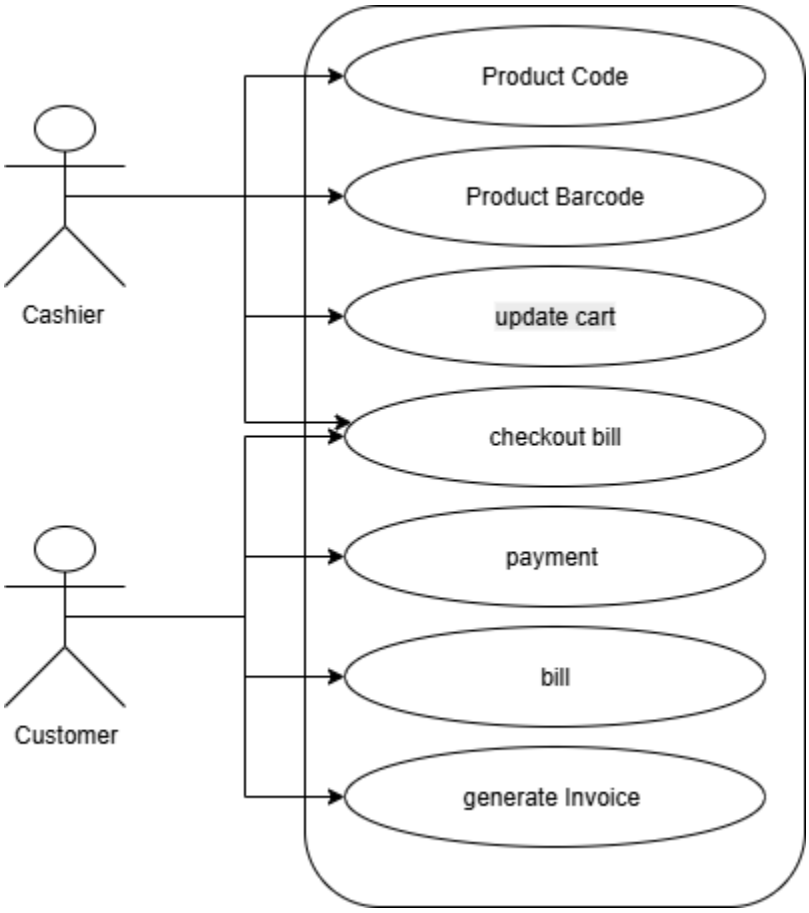


ER Daigram -

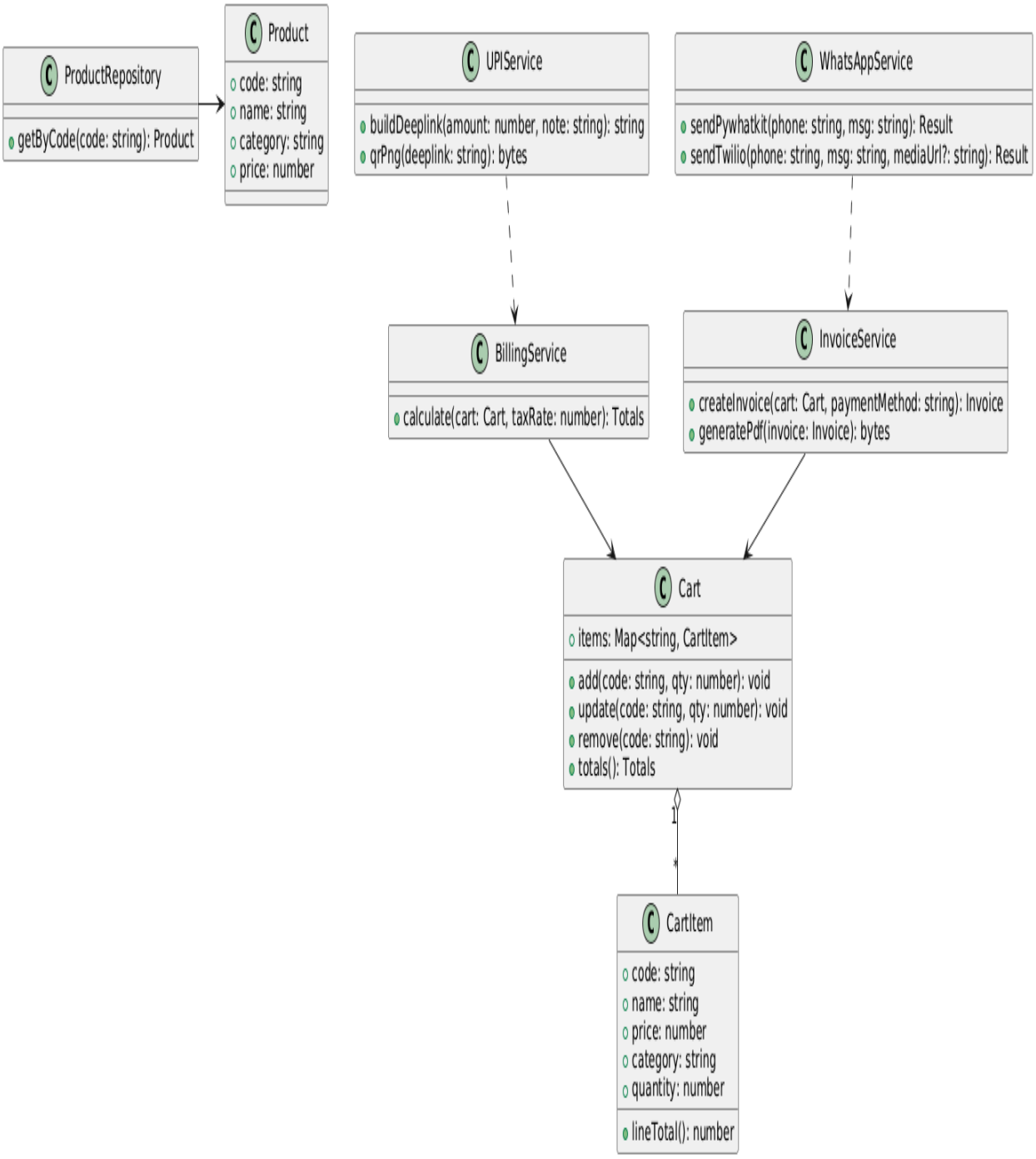

Customer
phone : string «PK»
name : string
email : string
places
Invoice
id : string «PK»
customerPhone : string «FK»
subtotal : decimal
tax : decimal
total : decimal
paymentMethod : string
createdAt : datetime
Product
code : string «PK»
name : string
category : string
price : decimal
has
contains
included in
Payment
id : string «PK»
invoiceId : string «FK»
method : string
status : string
transactionRef : string
createdAt : datetime
InvoiceItem
id : string «PK»
invoiceId : string «FK»
productCode : string «FK»
price : decimal
quantity : int
lineTotal : decimal

UML Diagram -

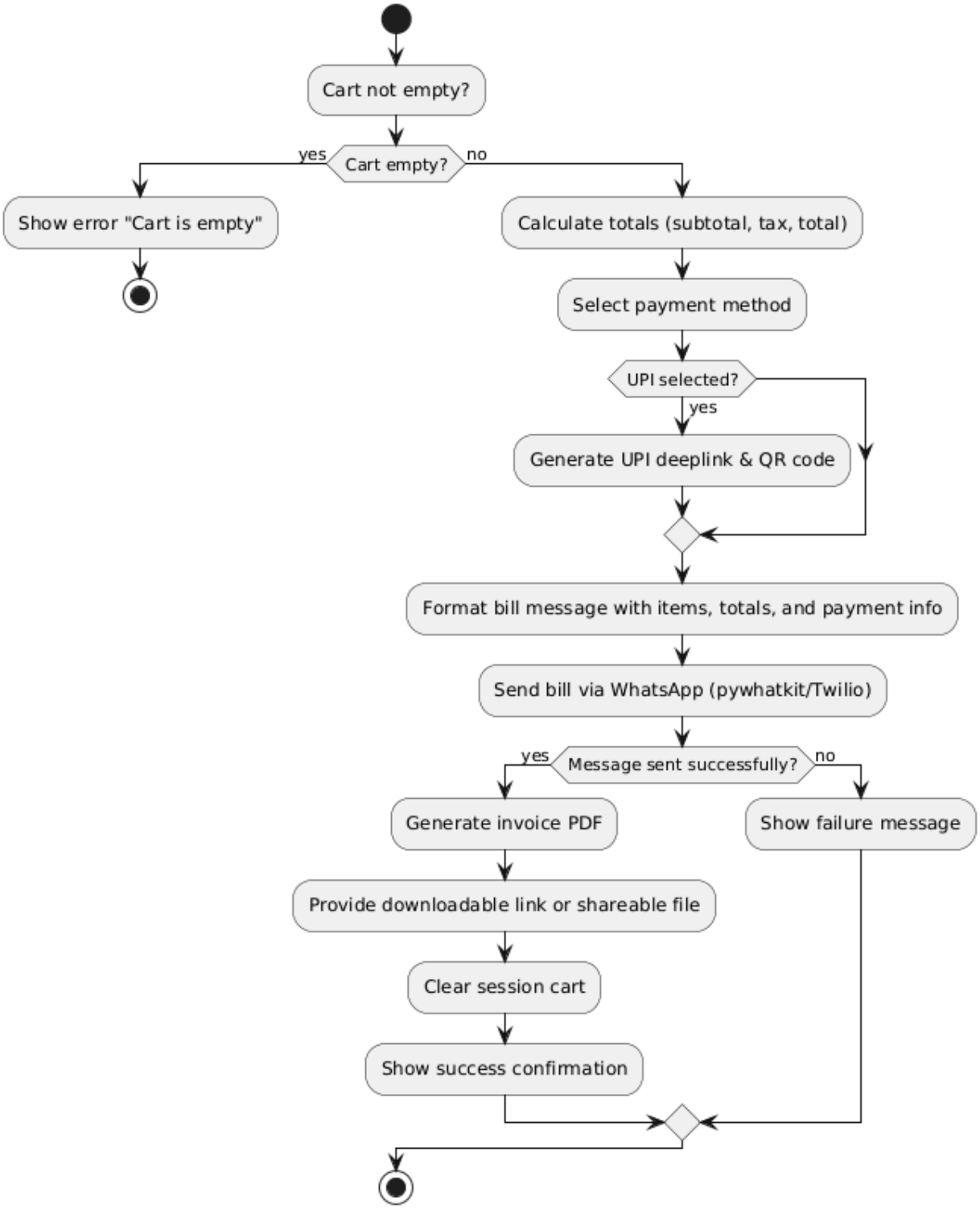
1. Use case diagram



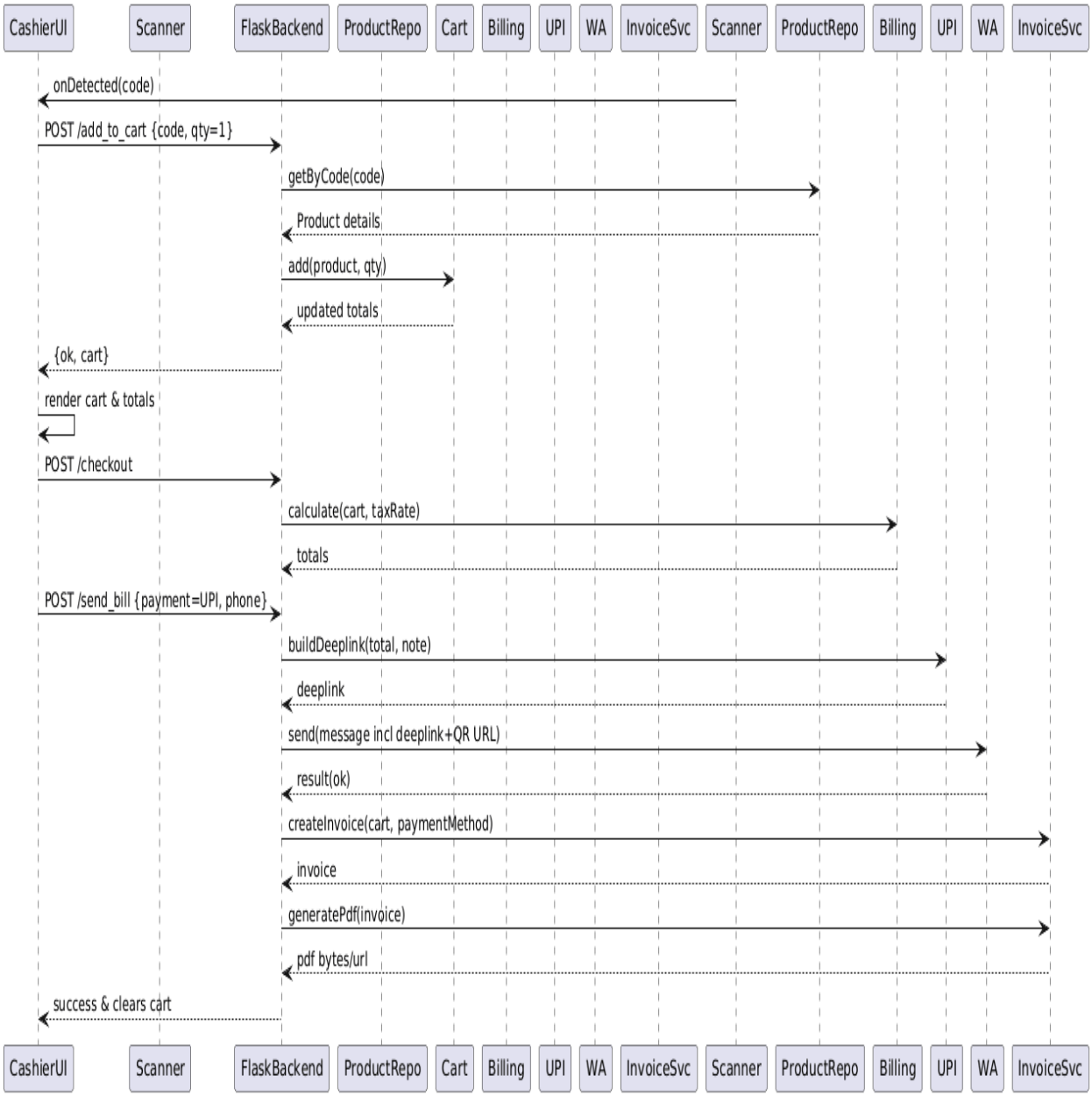
2. Class Diagram



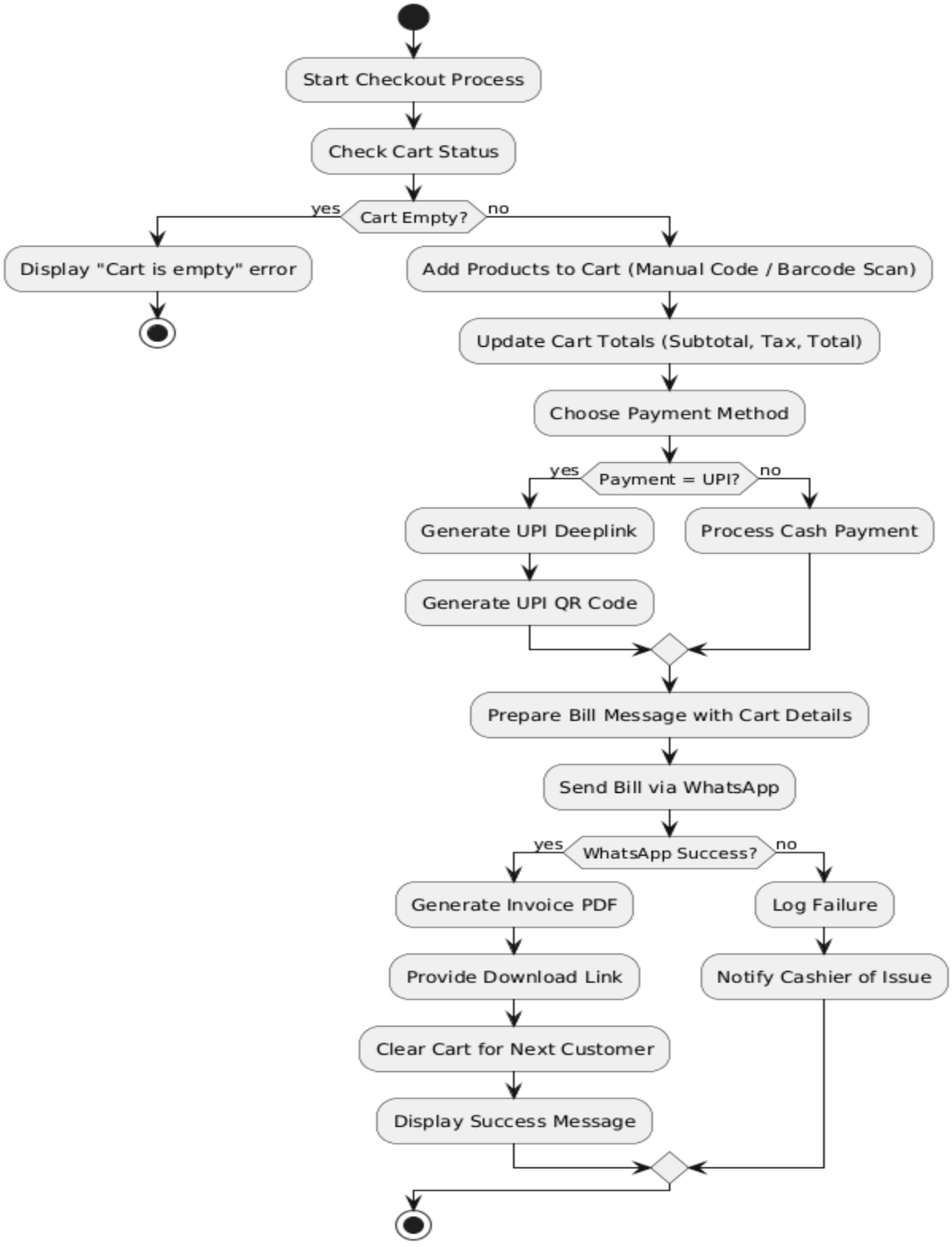
3. Activity Daigram



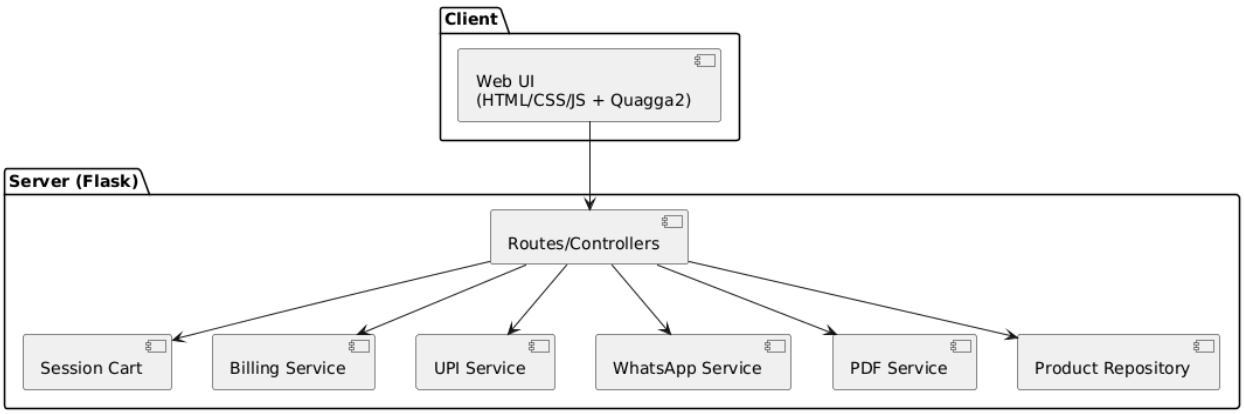
3. Sequence diagram



4. Flow chart


Start Checkout Process
Check Cart Status
Cart Empty?
yes
no
Display "Cart is empty" error
Add Products to Cart (Manual Code / Barcode Scan)
Update Cart Totals (Subtotal, Tax, Total)
Choose Payment Method
Payment = UPI?
yes
no
Generate UPI Deeplink
Process Cash Payment
Generate UPI QR Code
Prepare Bill Message with Cart Details
Send Bill via WhatsApp
WhatsApp Success?
yes
no
Generate Invoice PDF
Log Failure
Provide Download Link
Notify Cashier of Issue
Clear Cart for Next Customer
Display Success Message

5. Component Diagram



DATABASE TABLES -

Product Table -

Field Name	Data Type	Key / Constraint	Description
Code	TEXT	Primary Key	Unique product code
name	TEXT	Not null	Product name
category	TEXT		Product category
Price	Decimal	Not null	Price per unit

Customer Table -

Field Name	Data Type	Key / Constraint	Description
phone	TEXT	Primary Key	Customer phone number (unique ID)
name	TEXT		Full name of customer
email	TEXT	Unique	Email address of customer

Invoices Table -

Field Name	Data Type	Key / Constraint	Description
id	TEXT	Primary Key	Unique invoice ID
customerPhone	TEXT	Foreign Key	References Customers(phone)
subtotal	DECIMAL	NOT NULL	Total before tax
tax	DECIMAL	NOT NULL	Tax amount (GST/VAT)
total	DECIMAL	NOT NULL	Final invoice total
paymentMethod	TEXT	NOT NULL	Cash / UPI
createdAt	TIMESTAMP	NOT NULL	Invoice creation timestamp

InvoiceItems Table

Field Name	Data Type	Key / Constraint	Description
id	TEXT	Primary Key	Unique line item ID
invoiceId	TEXT	Foreign Key	References Invoices(id)
productCode	TEXT	Foreign Key	References Products(code)
price	DECIMAL	NOT NULL	Price per unit at time of sale
quantity	INTEGER	NOT NULL	Number of units sold
lineTotal	DECIMAL	NOT NULL	price × quantity

Payments Table

Field Name	Data Type	Key / Constraint	Description
id	TEXT	Primary Key	Unique payment ID
invoiceId	TEXT	Foreign Key	References Invoices(id)
method	TEXT	NOT NULL	Cash / UPI
status	TEXT	NOT NULL	Paid / Pending / Failed
transactionRef	TEXT		UPI transaction reference or receipt number
createdAt	TIMESTAMP	NOT NULL	Payment timestamp

Advantages -

1. **Automated Billing**
   Eliminates manual calculation errors by automating product entry, cart totals, tax computation, and final billing.

2. **Time-Saving**
   Checkout process is faster with live barcode scanning and instant cart updates, improving efficiency for cashiers and reducing customer wait times.

3. **Accuracy & Reliability**
   All calculations, totals, and taxes are computed server-side, ensuring consistency and reducing human error.

4. **Digital Payment Integration**
   Supports UPI payments with auto-generated deeplinks and QR codes, simplifying cashless transactions.

5. **WhatsApp Bill Delivery & PDF Invoice**
   Bills can be sent instantly to customers via WhatsApp, and professional PDF invoices are generated for records, making billing transparent and auditable.

6. **User-Friendly Interface**
   The web-based GUI is intuitive and responsive, allowing cashiers to manage products, payments, and invoices without technical expertise.

7. **Offline & Local-First Functionality**
   Works without requiring constant internet connectivity; session-based cart ensures uninterrupted operation.

8. **Extensible Architecture**
   Modular design allows easy future enhancements such as persistent inventory, discounts, multi-register synchronization, and cloud deployment.

Limitations -

1. **Hardware Dependency**
   Requires a functioning camera for barcode scanning; poor camera quality or lighting may reduce scanning efficiency.

2. **Performance with Large Inventory**
   System may slow down when handling a very large number of products or concurrent sessions (optimizations can be applied later).

3. **Internet Requirement for Digital Delivery**
   WhatsApp messaging and UPI scanning require internet connectivity; offline mode cannot send bills digitally.

4. **Manual Product Registration**
   All products must be pre-registered in the JSON database with correct codes, names, and prices before usage.

5. **Limited Multi-User Support**
   Current session-based cart does not support multi-cashier simultaneous operation without database integration.

6. **No Hardware Printer Integration**
   Receipts cannot be printed directly on thermal or POS printers in the demo version; only PDF generation is available.

FUTURE ENHANCEMENT -

1. **Cloud Integration & Multi-Device Support**
   Migrate the product and invoice database from local JSON/SQLite to cloud-based solutions, allowing multiple cashiers and devices to access and manage sales data in real-time.

2. **Mobile Application Development**
   Develop Android/iOS apps for cashiers and store managers to process bills, track sales, and generate invoices remotely.

3. **Inventory & Catalog Management**
   Add a full-featured inventory management module, including stock updates, product additions/deletions, and bulk import/export for efficient store operations.

4. **Discount & Promotions Engine**
   Introduce automated discount calculations, promotional offers, and loyalty points to enhance customer engagement.

5. **Thermal Printer & POS Integration**
   Enable direct printing of receipts using thermal or POS printers for seamless in-store operations.

6. **AI-Powered Analytics & Sales Insights**
   Implement AI/ML algorithms to analyze sales patterns, predict high-demand products, optimize stock levels, and generate business performance reports.

7. **Voice Command Integration**
   Introduce a voice-enabled interface for cashiers to add products, checkout, or send bills hands-free.

8. **Web-Based Dashboard for Management**
   Provide a centralized web dashboard for store managers to monitor sales, track payments, generate reports, and manage multiple billing counters.

9. **Integration with Payment Gateways**
   Expand UPI support to other digital payment methods (e.g., cards, wallets) and automate reconciliation with accounting systems.

10. **Notification & Alert System**
    Implement SMS/Email/Push notifications to inform customers about bills, promotions, or payment confirmations.

11. **Data Backup & Recovery System**
    Add automatic backup and recovery mechanisms to prevent loss of sales and invoice data in case of hardware/software failure.

12. **Scalability for Large Retail Operations**
    Optimize system architecture to handle multiple billing counters, large inventories, and high-volume transactions across multiple store locations.

CONCLUSION -

The **Supermarket Billing System** is a modern, efficient, and user-friendly point-of-sale solution designed to streamline retail checkout operations. By integrating live barcode scanning, manual product code entry, and automated cart and billing calculations, the system ensures fast, accurate, and reliable billing while minimizing human error. Its modular architecture, session-based cart management, and professional web GUI enhance usability for cashiers and store managers, while PDF invoice generation and WhatsApp delivery maintain record-keeping and customer communication efficiency.

The system's automation of product entry, tax calculation, and payment processing not only saves time but also improves reliability, accountability, and transparency in sales operations. While limitations such as camera quality for barcode scanning, local database dependency, and initial product registration exist, the system's design ensures smooth performance, and future enhancements like cloud integration, multi-device support, and AI-driven analytics will further strengthen its capabilities.

Overall, the **Supermarket Billing System** represents a significant step forward in digitizing retail billing processes. With continued development and integration of advanced features, it has the potential to transform small to medium retail operations, making them faster, more accurate, customer-friendly, and technology-driven, while laying a strong foundation for scalable, real-world POS deployments.

BIBLOGRAPHY -

Books & Personal Communication:

- Prof. Jayshree Mam – Personal Communication.

For further insights and guidance, reaching out to senior students who have experience with similar projects can be invaluable. They can provide practical advice and share lessons learned, which can help in refining the system and overcoming potential challenges. Engaging with these experienced peers could also open doors to new ideas and collaborations.

Online References:

- OpenAI. "Content Creation" – www.openai.com/chatgpt.
- Bolt.new. "Diagram & Visualization Tool" – www.bolt.new.
- PlantUML. "UML Diagrams & System Design" – www.plantuml.com.
- Instagram. "New Idea and Modification" – www.instagram.com.
- WhatsApp. "Messaging and Communication" – www.whatsapp.com.
- Mozilla Firefox. "Firefox Browser" – www.mozilla.org/firefox.
- Google Docs. "Online Document Collaboration" – www.docs.google.com.
- Visual Studio Code. "Website Development" – www.code.visualstudio.com.

Barcodes -


P1001


P1006


P1002


P1007


P1003


P1008


P1004


P1009


P1005


P1010